

07 기능 구현 Flask

1. 로그인

사용자 인증 및 세션 관리

실시간 운영 중

컨베이어 라인
중앙 관제 시스템

MFC 에지 컨트롤러 연동, 불량 감지,
센서 모니터링 및 비상정지 제어를
통합 관리합니다.

현재 접속자1명

금일 생산량1,284 개

불량률2.3%

현재 온도24.1 °C

© 2026 스마트 컨베이어 시스템 · v2.0.1

로그인

계정 정보를 입력하여 시스템에 접속하세요

이미 다른 기기에서 로그인 중입니다.
기존 세션을 종료하고 여기서 로그인하시겠습니까?

취소

강제 로그인

아이디

admin

비밀번호

....

☐ 로그인 상태 유지

로그인

SSL 암호화 통신 보안 인증 적용

127.0.0.1:5000의 메시지
다른 기기에서 로그인되어 현재 세션이 종료됩니다.
확인

DB에서 사용자 정보를 조회하여 로그인 가능 여부 판단

```
member = Member.query.get(user_id)
if not member or member.password != password:
    return jsonify({'status': 'error', 'message': '아이디 또는 비밀번호 불일치'}), 401
```

1차) 중복 로그인 감지 → 유저에게 선택권 제공

```
if is_user_online(member.id):
    if not force:

        print(f"[AUTH] 중복 로그인 감지: {member.id} (유저 선택 대기)")
        return jsonify({
            'status': 'duplicate_login',
            'message': '이미 다른 기기에서 로그인 중입니다.'
        }), 409
```

2차) 사용자 선택 (유지 / 강제 로그인)

```
else:

    print(f"[AUTH] 강제 로그인 요청: {member.id} → 기존 세션 종료")
    _force_logout_via_socket(member.id)
    _checkout(member.id)
    unregister_session(member.id)
```

07 기능 구현 Flask

2. 대시보드 (센서 데이터)



WebSocket 기반 실시간 데이터 전송
Socket.IO 기반 실시간 데이터 전송 (WebSocket 사용)

센서 → Flask → Socket.IO → 웹 대시보드

```
@api_bp.route('/status', methods=['POST'])
def api_post_status():

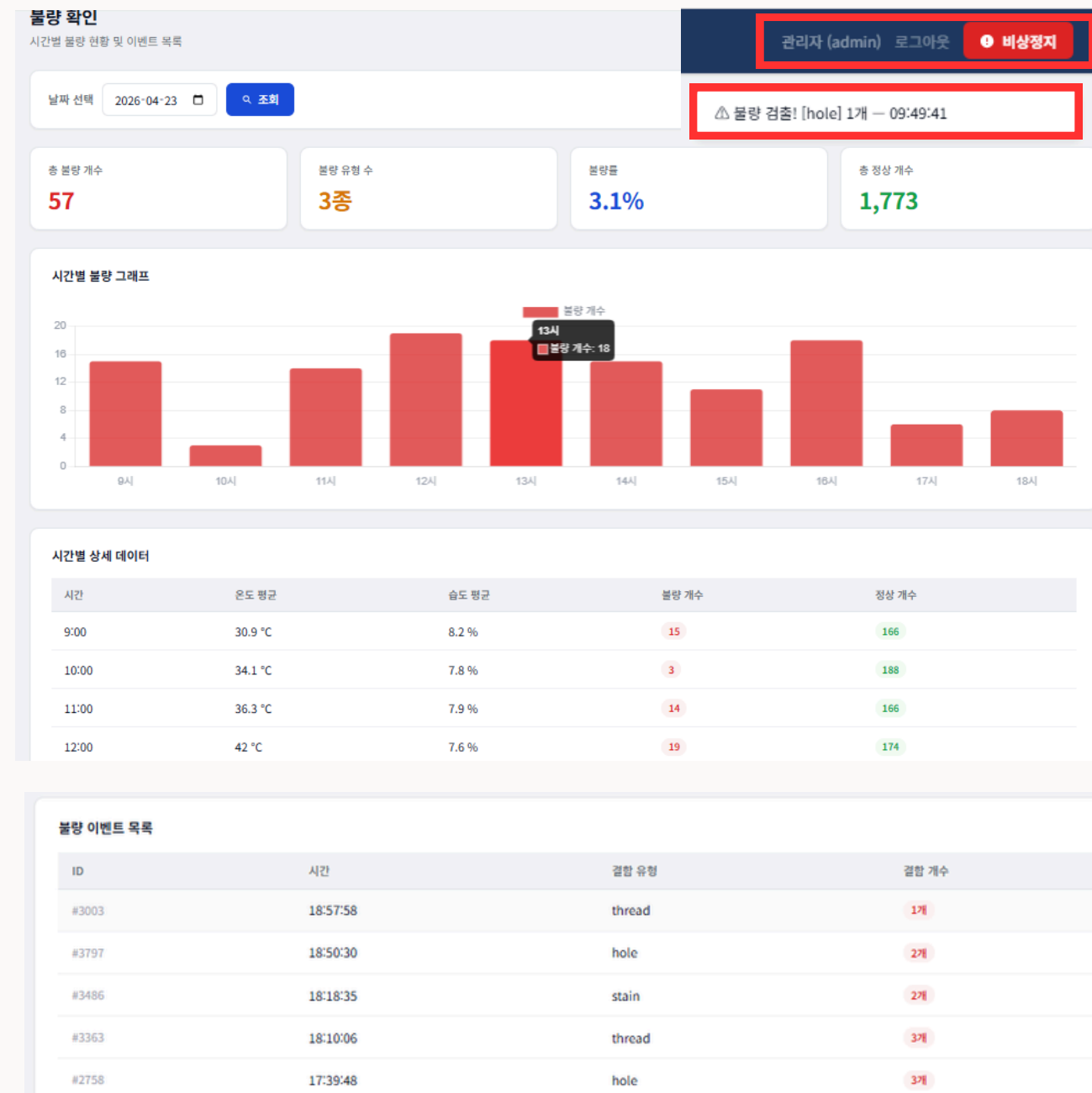
    if _socketio:
        _socketio.emit('sensor_update', {
            'temp': _mfc_status['temp'],
            'humidity': _mfc_status['humi'],
            'timestamp': _mfc_status['updated_at']
        })

        _socketio.emit('conveyor_status', {
            'status': 'on' if conveyor_on else 'off',
            'speed': _mfc_status['speed'],
            'fan': _mfc_status['fan'],
            'emg_stop': _mfc_status['emg_stop'],
            'led_state': led_state,
            'timestamp': _mfc_status['updated_at']
        })
```

← 실시간 데이터 전송
← 센서 데이터
← 센서 데이터 수집 시각
← 장비 상태까지 전송

07 기능 구현 Flask

3. 불량 결과 수신 및 전송



C#에서 검사 결과(정상/불량) 전송 → Flask에서 수신 → DB 저장 → 통계 및 (불량 시) 장비 제어

```
@api_bp.route('/defect', methods=['POST'])  
def new_defect():
```

↓ 데이터 저장

```
try:  
    db.session.add(event)  정상/불량 여부를 포함한 모든 검사 결과를 DB에 저장  
    db.session.commit()
```

↓ 장비 제어

불량 데이터일 경우에만 현장관리자에게 전달

```
if defect_type != 'normal': ← 핵심 조건  
    try:  
        from tcp_server import send_defect_servo  
        send_defect_servo() ← 실제 실행  
    except Exception:
```

검사 결과는 DB에 저장되고, 불량일 경우에만 장비 제어가 수행됨

07 기능 구현 Flask

4. 영상 프레임 및 판별 결과 수신

C#에서 프레임 단위 영상과 판별 결과를 함께 수신하여
웹으로 실시간 전달

```
@app.route('/predict', methods=['POST'])
def predict():
```



```
img_base64 = data['image_data']
```



```
socketio.emit('video_frame', {'frame': img_base64})
```

C# (영상 분석) → Flask /predict → Socket.IO → 웹 화면 출력

5. TCP로 MFC에 명령 전송

불량 발생 시 실제 장비를 제어하는 통신

```
def broadcast_to_mfc(message):
```

 실제 장비로 명령 보내는 부분

⋮

```
for client_sock in _mfc_clients:
```

```
try:
```

```
    client_sock.sendall(message.encode('utf-8'))
```

 ← 실제 전송

```
    print(f"[TCP] ✓ MFC에 전송 성공: {message.strip()}")
```

```
    sent += 1
```

```
except Exception as e:
```

```
    print(f"[TCP] X MFC 전송 실패: {e}")
```

```
dead.append(client_sock)
```

↓ TCP 전송

```
def send_defect_servo():
```

```
    broadcast_to_mfc("DEFECT:SERVO\n")
```

 불량 발생 시 실행되는 명령

검출 → 판단 → 물리적 제어까지 이어지는 자동화 시스템 구현

07 기능 구현 Flask

6. 모니터링 / 웹 영상 출력

모니터링
실시간 영상 스트림

LIVE10 FPS

DEFECTMODE: WAIT SAMPLE OUT 0/8 [bg=True hsv_area=0]

hole 0.97

thread 0.84

Defect Rate: 90.9% (10/11)

스트리밍 중

스트림 정보

상태

스트리밍 중

수신 프레임

지금까지 받은
총 영상 프레임 수

8,552

평균 FPS

초당 프레임 처리

10 FPS

마지막 수신

10:29:12

수신 로그

[10:29:12] 프레임 #8550 수신

[10:29:11] 프레임 #8540 수신

[10:29:10] 프레임 #8530 수신

[10:29:09] 프레임 #8520 수신

[10:29:08] 프레임 #8510 수신

[10:29:07] 프레임 #8500 수신

[10:29:06] 프레임 #8490 수신

[10:29:05] 프레임 #8480 수신

[10:29:04] 프레임 #8470 수신

총 불량 개수

9

총 정상 개수

5

WebSocket으로 영상 프레임 실시간 수신

C#에서 전달된 영상을 Socket.IO로 수신하여 웹에 표시

```
socket.on('video_frame', function(data) {  
    const frame = data.frame;  
    if (!frame) return;  
  
    const img = document.getElementById('liveFrame');  
    const noSignal = document.getElementById('noSignal');  
  
    // 첫 프레임 수신 시  
    if (!isStreaming) {  
        isStreaming = true;  
        img.style.display = 'block';  
        noSignal.style.display = 'none';  
        document.getElementById('live-dot').classList.add('live');  
        setConnected();  
    }  
  
    img.src = 'data:image/jpeg;base64,' + frame;
```

Flask는 영상 분석을 수행하지 않고, 전달된 영상을 화면에 출력하는 역할만 수행

07 기능 구현 Flask

7. 회원 관리

관리자 권한 기반 사용자 관리 HTTP 요청(fetch) → Flask → DB 처리 → JSON 응답 → 목록 갱신

회원 관리

회원 추가

아이디 *

비밀번호 *

이름

연도-월-일

직급

+ 추가

회원 목록

5명

회원	아이디	생년월일	직급	권한	가입일	관리
관 관리자	admin	1995-03-15	관리자	관리자	2026-04-24	수정 삭제
작 작업자1	user1	2026-04-22	라인 A 담당	일반	2026-04-24	수정 삭제
작 작업자2	user2	2026-04-22	라인 B 담당	일반	2026-04-24	수정 삭제
현 현장 관리자1	user3	2026-04-22	설비 총괄	일반	2026-04-24	수정 삭제

작업자1 (user1) 로그아웃

🔒

접근 권한 없음

회원 관리는 관리자만
이용할 수 있습니다

이 페이지는 관리자 권한이 필요합니다.
접근이 필요하다면 관리자에게 문의하세요.

← 대시보드로 돌아가기

1) 회원 목록 불러오기

```
function loadMembers() {  
  fetch('/api/members')
```

DB → Flask → 웹

2) 회원 추가

```
function addMember() {  
  fetch('/api/member/add', {  
    method: 'POST', headers:
```

사용자 입력 → Flask → DB 저장

3) 회원 수정

```
fetch('/api/member/update', {  
  method: 'POST',  
})
```

4) 회원 삭제

```
function deleteMember(id) {  
  fetch('/api/member/delete', {  
    method: 'POST',
```

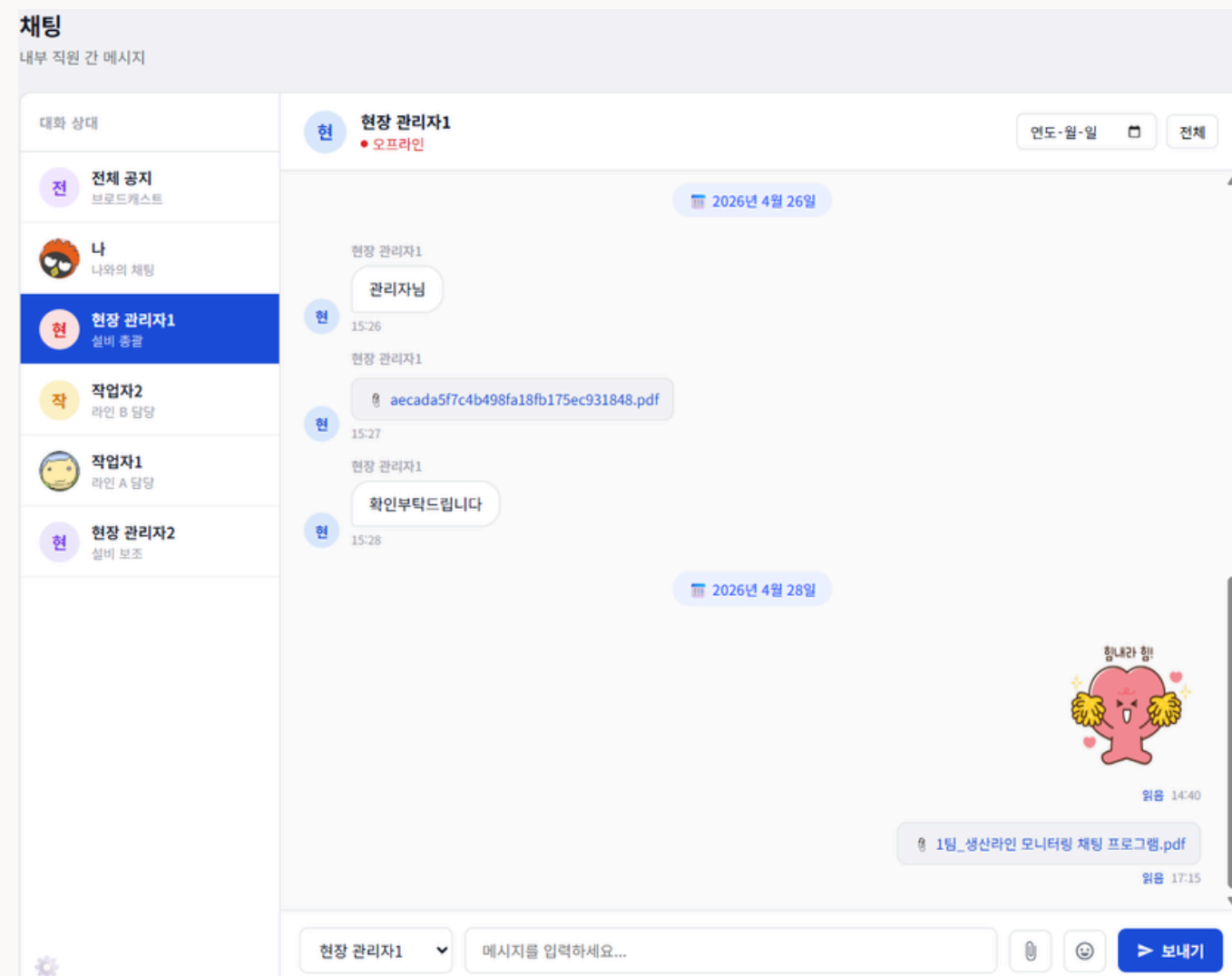
5) 회원 목록 갱신

```
loadMembers();  
['m-id', 'm-pw', 'm-name', 'm-birth', 'm-position'].forEach(id => {  
  showToast('success', res.message || '회원이 추가되었습니다.');
```

07 기능 구현 Flask

8. 실시간 채팅

작업자간 실시간 소통 기능 제공



작업자 → Flask → Socket.IO → 상대 사용자
→ 화면 표시 WebSocket 기반 실시간 메시지 전달

1) 채팅 전송

```
function sendMsg() {  
  const msg = input.value.trim();  
  
  socket.emit('chat_message', {  
    receiver_id: receiverId,  
    message: msg  
  });  
}
```

작업자 → Flask → Socket.IO 보내는 코드

2) 채팅 수신

```
socket.on('receive_message', function(data) {  
  const selectedId = document.getElementById('recipient').value;
```

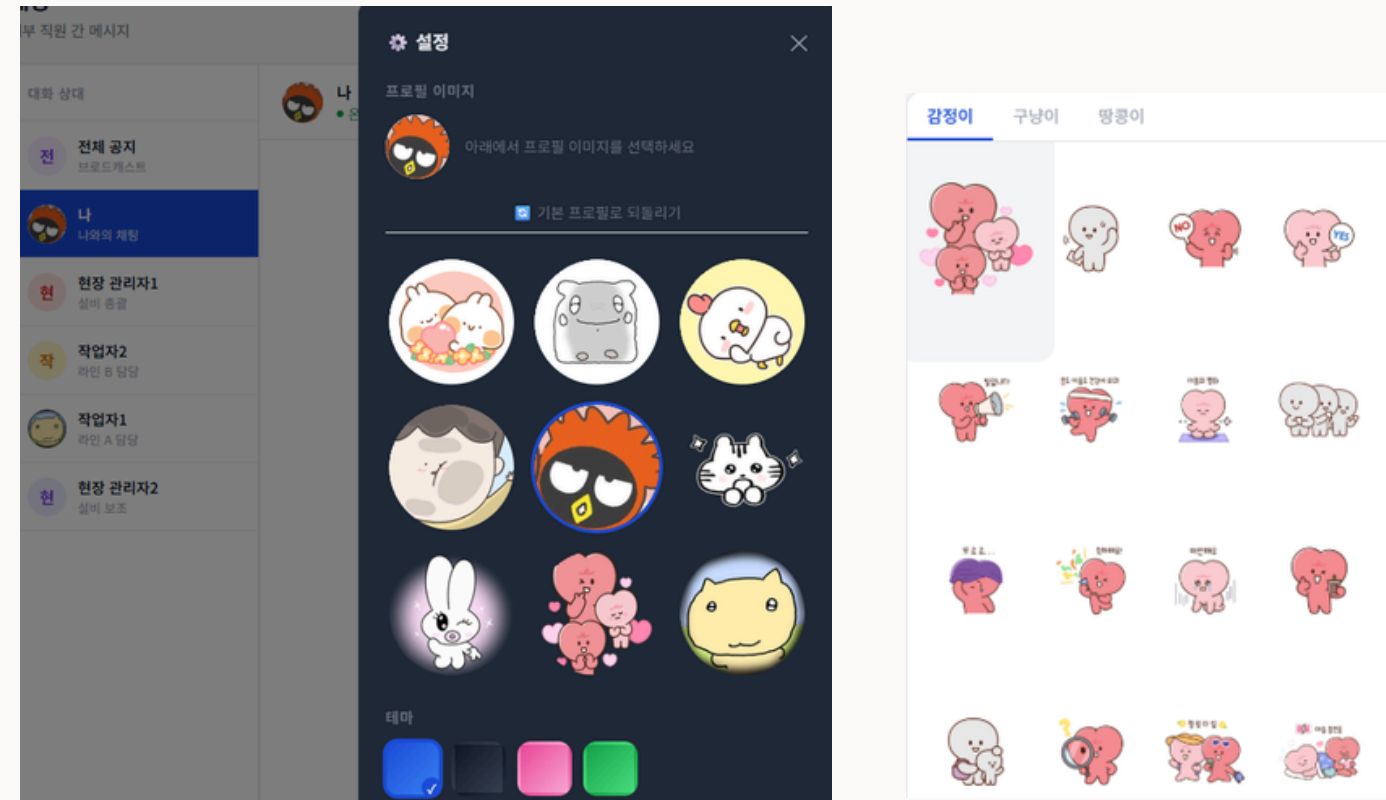
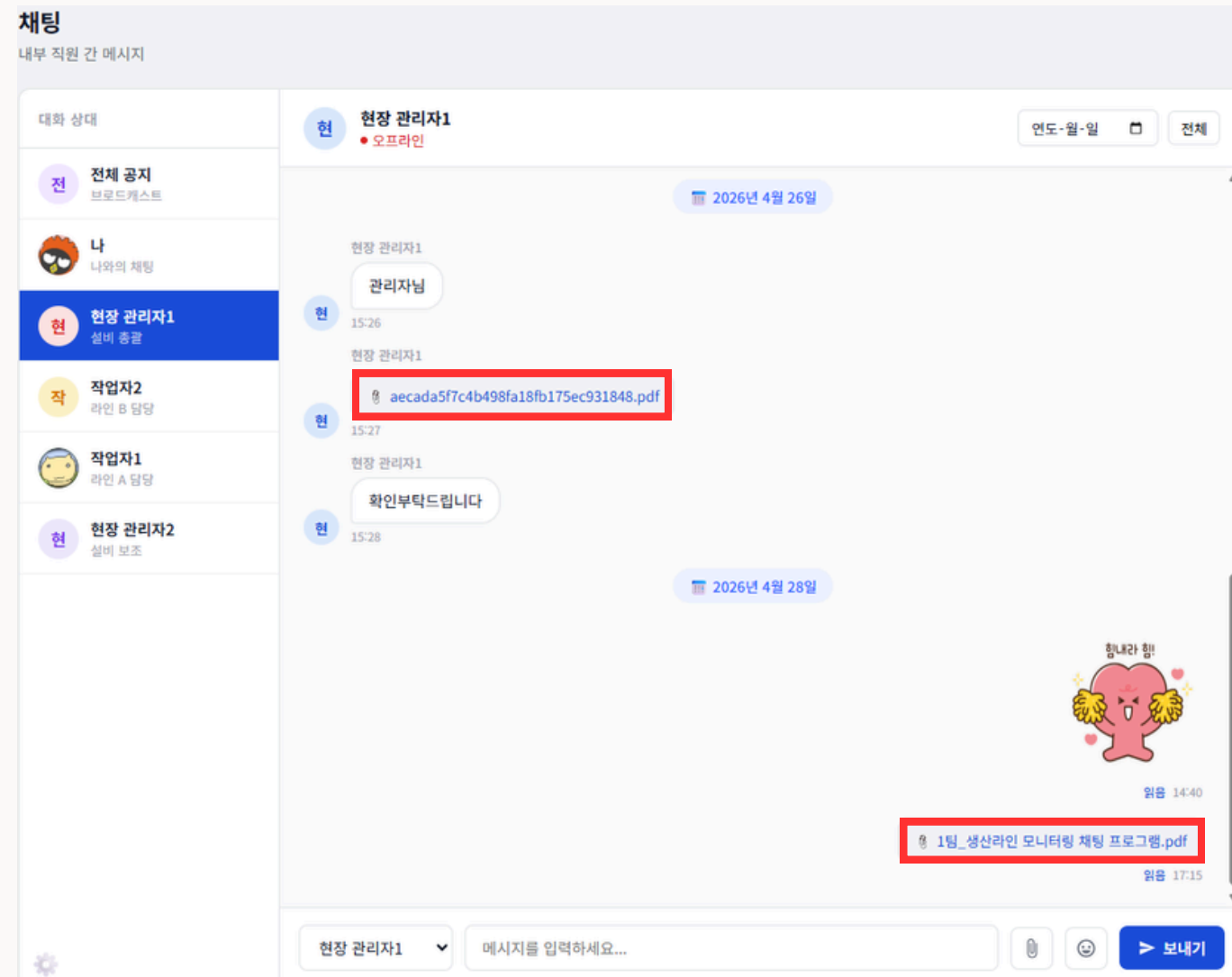
Flask → 작업자 (메시지 받는 코드)

3) 채팅 화면 출력

```
container.appendChild(el);  
function addMessageToScreen(data) {
```

07 기능 구현 Flask

8. 실시간 채팅



사용자 프로필 이미지 및 이모티콘 기반 채팅 UI 구성

파일 전송

```
const formData = new FormData();
formData.append('file', file);
formData.append('receiver_id', receiverId);

fetch('/api/chat/upload', {
  method: 'POST',
  body: formData
})
```

파일 업로드 요청(HTTP) → Flask → 서버 저장 후 상대 사용자에게 전달